

Repaso General I Parcial

Asignatura: Fundamentos de la Programación

Docente: Ing. Raúl Fernández Fajardo



Repaso General I Parcial

Asignatura: Fundamentos de la Programación

Docente: Ing. Raúl Fernández Fajardo

Objetivo General

Repasar y reforzar los conceptos fundamentales del primer parcial, aplicando el pensamiento lógico, el uso de variables, estructuras de control (if, switch, for, while, do...while) y operadores, mediante ejercicios prácticos y análisis guiado de código.

Objetivos Específicos

- Revisar y aplicar el uso de las estructuras de control condicionales, repetitivas y de salto en la resolución de problemas básicos en C++.
- Fortalecer la lógica algorítmica mediante ejercicios con pseudocódigo, ciclos personalizados, y estructuras if y switch, identificando errores comunes de sintaxis y estructura.

Repaso rápido

Cuadro: Pensamiento Lógico, Algoritmo y Pseudocódigo

 Concepto

 Definición / Explicación

Pensamiento Lógico

Es la capacidad de analizar, descomponer, planificar y resolver problemas paso a paso. Base esencial en la programación.

Fases del Pensamiento Lógico

1. Conocer el problema.
2. Dividirlo en partes pequeñas.
3. Planificar cómo resolver cada parte.
4. Escribir el código y verificar.

Algoritmo

Es una secuencia ordenada de pasos lógicos y finitos para resolver un problema o realizar una tarea específica.

Pseudocódigo

Es una forma de representar un algoritmo usando lenguaje natural estructurado, más fácil de entender que el código real.

Diferencia entre algoritmo y pseudocódigo

El algoritmo es la idea o lógica en sí misma; el pseudocódigo es una forma de escribir ese algoritmo de manera legible.



¿Cómo se relacionan?

- Usamos el **pensamiento lógico** para entender y resolver problemas.

- La **solución se diseña como un algoritmo** (pasos ordenados).

- Antes de programar,

usamos **pseudocódigo para planificar** sin errores de sintaxis.

- Luego, se **implementa en código real**.

Repaso rápido

Tabla: Variables y Tipos de Datos en C++

Concepto

Descripción / Ejemplos

¿Qué es una variable?

Una variable es un espacio en memoria que almacena un valor que puede cambiar durante la ejecución del programa.

Tipos de Variables (por uso)

- **Local:** Declarada dentro de una función o bloque `{}`.
- **Global:** Declarada fuera de todas las funciones.
- **Constante:** Su valor no puede cambiar. Se declara con `const`.

Tipos de Datos (comunes en C++)

- `int` → Números enteros: `5`, `-10`
- `float` → Decimales: `3.14`
- `double` → Decimales de mayor precisión
- `char` → Un carácter: `'A'`, `'7'`
- `bool` → Booleano: `true` o `false`
- `string` → Texto (requiere `#include <string>`)

"Cada variable es una pequeña caja de posibilidades. Aprende a nombrarlas, entenderlas y usarlas, y podrás construir cualquier programa que imagines."

```
int edad = 20;           // Variable entera
float pi = 3.14;         // Decimal simple
char letra = 'A';        // Caracter
bool activo = true;      // Valor booleano
string nombre = "Ana";   // Cadena de texto
```


Repaso rápido estructuras de control

Estructuras Condicionales

 Instrucción	 Descripción
<code>if</code>	Ejecuta un bloque si una condición es verdadera.
<code>if...else</code>	Ejecuta un bloque si la condición es verdadera, y otro si es falsa.
<code>else if</code>	Evalúa condiciones adicionales si el <code>if</code> inicial es falso.
<code>switch</code>	Selecciona entre múltiples casos posibles según el valor de una variable.

Estructuras de Salto

 Instrucción	 Descripción
<code>break</code>	Salte de un ciclo o <code>switch</code> .
<code>continue</code>	Salta al inicio de la siguiente iteración del ciclo.
<code>goto</code>	Salta a una etiqueta específica del código. ⚠ Poco recomendable
<code>return</code>	Salte de una función y puede devolver un valor.

Estructuras Repetitivas

 Instrucción	 Descripción
<code>for</code>	Repite un bloque de código un número conocido de veces.
<code>while</code>	Repite mientras se cumpla una condición.
<code>do...while</code>	Repite al menos una vez y luego mientras se cumpla la condición.

"Dominar las estructuras de control" es como aprender a dirigir el flujo de tus ideas. Cada instrucción es una decisión que transforma problemas en soluciones."

Calculadora simple con if y switch

```
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5      int opcion;
6      double num1, num2, resultado;
7
8      // Mostrar el menú por primera vez
9      cout << "\n==== CALCULADORA SIMPLE =====> endl;
10     cout << "1. Sumar" << endl;
11     cout << "2. Restar" << endl;
12     cout << "3. Multiplicar" << endl;
13     cout << "4. Dividir" << endl;
14     cout << "5. Salir" << endl;
15     cout << "Seleccione una opción (1-5): ";
16     cin >> opcion;
17
18     // Si la opción es diferente de 5, se continúa con las operaciones
19     if (opcion != 5) {
20
21         // Solicitar números solo si no eligió salir
22         cout << "Ingrese el primer número: ";
23         cin >> num1;
24         cout << "Ingrese el segundo número: ";
25         cin >> num2;
26
27         // Selección de operación con switch
28         switch (opcion) {
29             case 1:
30                 resultado = num1 + num2;
31                 cout << "Resultado: " << resultado << endl;
32                 cout << "Fin del Programa: " << resultado << endl;
33                 break;
```

```
34         case 2:
35             resultado = num1 - num2;
36             cout << "Resultado: " << resultado << endl;
37             cout << "Fin del Programa: " << resultado << endl;
38             break;
39         case 3:
40             resultado = num1 * num2;
41             cout << "Resultado: " << resultado << endl;
42             cout << "Fin del Programa: " << resultado << endl;
43             break;
44         case 4:
45             // Validar que el divisor no sea cero usando if
46             if (num2 != 0) {
47                 resultado = num1 / num2;
48                 cout << "Resultado: " << resultado << endl;
49                 cout << "Fin del Programa: " << resultado << endl;
50             } else {
51                 cout << "Error: No se puede dividir entre cero." << endl;
52                 cout << "Fin del Programa: " << resultado << endl;
53             }
54             break;
55         default:
56             // Si se eligió una opción inválida
57             cout << "Opción inválida." << endl;
58             cout << "Fin del Programa: " << resultado << endl;
59             break;
60     }
61
62     } else {
63         // Si el usuario eligió salir (opción 5)
64         cout << "Saliendo del programa. ¡Hasta luego!" << endl;
65     }
66
67     return 0;
68 }
```

Output

```
==== CALCULADORA SIMPLE =====
1. Sumar
2. Restar
3. Multiplicar
4. Dividir
5. Salir
Seleccione una opción (1-5): 5
Saliendo del programa. ¡Hasta luego!
```

Comentarios clave explicados:

- if (opcion != 5) → Si no eligió salir, se le pide que ingrese números y se evalúa la operación
- if (num2 != 0) → Asegura que no se divida entre cero antes de realizar la división.
- else → Muestra un mensaje de despedida si elige la opción 5 desde el principio.

Ciclos for personalizados con menú

```
1 #include <iostream>
2 using namespace std;
3
4 int main() {
5     int vueltasPrimerCiclo, vueltasSegundoCiclo;
6     int opcion;
7
8     // Menú principal
9     cout << "===== MENÚ PRINCIPAL =====" << endl;
10    cout << "Este programa usará dos ciclos for anidados." << endl;
11    cout << "Tú decides cuántas vueltas tendrá cada uno." << endl;
12
13    // Pedir al usuario cuántas vueltas quiere en cada ciclo
14    cout << "\nIngrese cuántas vueltas desea en el primer ciclo: ";
15    cin >> vueltasPrimerCiclo;
16
17    cout << "Ingrese cuántas vueltas desea en el segundo ciclo: ";
18    cin >> vueltasSegundoCiclo;
19
20    // Confirmar opción antes de ejecutar
21    cout << "\nPresione 1 para ejecutar los ciclos o cualquier otro número para salir: ";
22    cin >> opcion;
23
24    if (opcion == 1) {
25        // Ciclo externo (primer ciclo)
26        for (int i = 1; i <= vueltasPrimerCiclo; i++) {
27            cout << "Vuelta " << i << " del primer ciclo" << endl;
28
29            // Ciclo interno (segundo ciclo)
30            for (int j = 1; j <= vueltasSegundoCiclo; j++) {
31                cout << "    Vuelta " << j << " del segundo ciclo" << endl;
32            }
33        }
34    } else {
35        // Si el usuario no desea ejecutar
36        cout << "Saliendo del programa. ¡Hasta pronto!" << endl;
37    }
38
39    return 0;
40 }
```

Output

```
===== MENÚ PRINCIPAL =====
Este programa usará dos ciclos for anidados.
Tú decides cuántas vueltas tendrá cada uno.

Ingrese cuántas vueltas desea en el primer ciclo: 4
Ingrese cuántas vueltas desea en el segundo ciclo: 2

Presione 1 para ejecutar los ciclos o cualquier otro número para salir: 1
Vuelta 1 del primer ciclo
    Vuelta 1 del segundo ciclo
    Vuelta 2 del segundo ciclo
Vuelta 2 del primer ciclo
    Vuelta 1 del segundo ciclo
    Vuelta 2 del segundo ciclo
Vuelta 3 del primer ciclo
    Vuelta 1 del segundo ciclo
    Vuelta 2 del segundo ciclo
Vuelta 4 del primer ciclo
    Vuelta 1 del segundo ciclo
    Vuelta 2 del segundo ciclo
```

¿Qué se repasa con este código?

Concepto	Aplicación en el código
for simple	Repetición controlada por el usuario
for anidado	Comprensión del flujo interno y externo
Entrada por teclado (cin)	Interactividad con el usuario
Condicional if	Verifica si se debe ejecutar o salir del programa

🔔 ¡Importante Recordar!

En C++, por cada llave { que se abre, debe cerrarse con una }. Esto es esencial para definir correctamente los bloques de código. Debes asegurarte de cerrar correctamente las llaves en estructuras como:

- main() → contiene todo el cuerpo del programa.
- Ciclos (for, while, do...while) → encierran las instrucciones que se repiten.
- Condicionales (if, else if, else) → delimitan el bloque que se ejecutará si se cumple la condición.
- Estructuras switch → contienen los case y el bloque que se evalúa.

No cerrar una llave correctamente puede provocar **errores de compilación** o **comportamiento inesperado del programa**.



Una llave que abre, debe cerrar. Siempre.

Menú con while, do...while y salir

```
1 #include <iostream>
2 using namespace std;
3
4 int main() {
5     int opcion;
6
7     do {
8         // Menú principal
9         cout << "\n==== MENÚ PRINCIPAL =====> endl;
10        cout << "1. Ver ejemplo con ciclo while" << endl;
11        cout << "2. Ver ejemplo con ciclo do...while" << endl;
12        cout << "3. Salir" << endl;
13        cout << "Seleccione una opción (1-3): ";
14        cin >> opcion;
15
16        if (opcion == 1) {
17            // Ejemplo con ciclo while
18            cout << "\nEjemplo con ciclo while:" << endl;
19            int i = 1;
20            while (i <= 3) { // Mientras i sea menor o igual a 3
21                cout << "Vuelta " << i << " del ciclo while" << endl;
22                i++;
23            }
24        }
25
26        else if (opcion == 2) {
27            // Ejemplo con ciclo do...while
28            cout << "\nEjemplo con ciclo do...while:" << endl;
29            int j = 1;
30            do {
31                cout << "Vuelta " << j << " del ciclo do...while" << endl;
32                j++;
33            } while (j <= 3); // Se ejecuta al menos una vez, luego verifica la condición
34        }
35
36        else if (opcion == 3) {
37            // Salida del programa
38            cout << "Saliendo del programa. ¡Hasta luego!" << endl;
39        }
40
41        else {
42            // Opción inválida
43            cout << "Opción no válida. Intente nuevamente." << endl;
44        }
45    } while (opcion != 3); // El ciclo do...while mantiene el menú hasta que se elija salir
46
47    return 0;
48 }
```

```
===== MENÚ PRINCIPAL =====
1. Ver ejemplo con ciclo while
2. Ver ejemplo con ciclo do...while
3. Salir
```

Seleccione una opción (1-3): 1

Ejemplo con ciclo while:

Vuelta 1 del ciclo while

Vuelta 2 del ciclo while

Vuelta 3 del ciclo while

```
===== MENÚ PRINCIPAL =====
```

1. Ver ejemplo con ciclo while

2. Ver ejemplo con ciclo do...while

3. Salir

Seleccione una opción (1-3): 2

Ejemplo con ciclo do...while:

Vuelta 1 del ciclo do...while

Vuelta 2 del ciclo do...while

Vuelta 3 del ciclo do...while

```
===== MENÚ PRINCIPAL =====
```

1. Ver ejemplo con ciclo while

2. Ver ejemplo con ciclo do...while

3. Salir

Seleccione una opción (1-3): 3

Saliendo del programa. ¡Hasta luego!

No cerrar una llave correctamente puede provocar **errores de compilación o comportamiento inesperado del programa.**

 **Una llave que abre, debe cerrar. Siempre.**

💡 ¿Qué hace este programa? (Explicación sencilla)

- Muestra un menú principal repetidamente usando un `do...while`.
- Si eliges opción 1, muestra un ciclo `while` que imprime 3 vueltas.
- Si eliges opción 2, muestra un ciclo `do...while` que también imprime 3 vueltas.
- La diferencia es que el `do...while` siempre se ejecuta al menos una vez, incluso si la condición fuera falsa.
- Si eliges 3, el programa se cierra.
- Si escribes otra cosa, te dirá que la opción no es válida.

Repaso General I Parcial

Asignatura: Fundamentos de la Programación

Docente: Ing. Raúl Fernández Fajardo

Repaso General I Parcial

Asignatura: Fundamentos de la Programación

Docente: Ing. Raúl Fernández Fajardo

